

COURSE CODE/TITLE:	CPE212/Automating server and Management	SECTION:	CPE31S2
DATE PERFORMED:	8/18/26	DATE SUBMITTED:	8/18/26
NAME:	Javillonar, Madhel Kathlyn Ann A.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Hands-on Activity: Ansible Basics

1. PROCEDURE OUTPUT

Task 1: Run elevated ad hoc commands

1. Locally, we use the command ***sudo apt update*** when we want to download package information from all configured resources. The sources are often defined in ***/etc/apt/sources.list*** file and other files located in

```

Aug 18 09:55
wdjavillonar@workstation: ~
wdjavillonar@workstation:~$ sudo apt update
[sudo] password for wdjavillonar:
Hit:1 http://ph.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://ph.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:4 http://ph.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,240 kB]
Get:6 http://ph.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,098 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [1,330 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Component [1,330 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [1,330 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [304 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,201 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [240 kB]

```

/etc/apt/sources.list.d/ directory. So, when you run the update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or

their dependencies. We can only run an apt update command in a remote machine. Issue the following command:

ansible all -m apt -a update_cache=true

```
wdjavillonar@workstation:~$ ansible all -m apt -a update_cache=true
192.168.56.102 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory
/var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open
(13: Permission denied)"
}
192.168.56.101 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory
/var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open
(13: Permission denied)"
}
```

What is the result of the command? Is it successful? The command failed because updating the package cache requires administrative privileges and ansible was unable to execute the apt module on the remote servers without sudo access.

Try editing the command and add something that would elevate the privilege. Issue the command *ansible all -m apt -a update_cache=true --become --ask-become-pass*. Enter the sudo password when prompted. You will notice now that the output of this command is a success. The *update_cache=true* is the same thing as running `sudo apt update`. The *--become* command elevates the privileges and the *--ask-become-pass* asks for the password. For now, even if we only changed the packaged index, we were able to change something on the remote server. You may notice after the second command was executed, the status is

```
wdjavillonar@workstation:~$ ansible all -m apt -a update_cache=true --beco
sk-become-pass
BECOME password:
192.168.56.101 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787018499,
    "cache_updated": true,
    "changed": true
}
192.168.56.102 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787018507,
    "cache_updated": true,
    "changed": true
}
```

CHANGED compared to the first command, which is **FAILED**.

```
ubuntu1 | FAILED! => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1786975175,
    "cache_updated": true,
    "changed": true
}
```

2. Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction. Here is the command: ***ansible all -m apt -a name=vim-nox --become --ask-become-pass***. The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.

```
wdjavillonar@workstation:~$ ansible all -m apt -a "name=vim-nox" --become-passBECOME password:
192.168.56.102 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787018507,
  "cache_updated": false,
  "changed": false
}
192.168.56.101 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787018499,
  "cache_updated": false,
  "changed": false
}
wdjavillonar@workstation:~$
```

- a. Verify that you have installed the package in the remote servers. Issue the command *which vim*

```
wdjavillonar@workstation:~$ ansible all -m shell -a "which vim"
192.168.56.101 | CHANGED | rc=0 >>
/usr/bin/vim
192.168.56.102 | CHANGED | rc=0 >>
/usr/bin/vim

wdjavillonar@workstation:~$ ansible all -m shell -a "apt search vim-nox | head -20"
192.168.56.101 | CHANGED | rc=0 >>
Sorting...
Full Text Search...
vim-nox/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.18 amd64 [i
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.18 amd64 [i
  Vi IMproved - enhanced vi editor - compact version
192.168.56.102 | CHANGED | rc=0 >>
Sorting...
Full Text Search...
vim-nox/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.18 amd64 [i
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.18 amd64 [i
  Vi IMproved - enhanced vi editor - compact version
```

and the command *apt search vim-nox* respectively. Was the command successful?

- Yes, the command was successful. The output shows the vim-nox package information on the remote servers, confirming that the package is available and installed.

b. Check the logs in the servers using the following commands: *cd /var/log*. After this, issue the command *ls*, go to the folder *apt* and open *history.log*. Describe what you see in the *history.log*.

```

jdjvillonar@workstation:~$ ansible all -m shell -a "cd /var/log && ls"
192.168.56.101 | CHANGED | rc=0 >>
alternatives.log
appport.log
appport.log.1
apt
auth.log
auth.log.1
boot.log
boot.log.1
bootstrap.log
dtmp
cloud-init.log
cloud-init-output.log
cups
cups-browsed
dist-upgrade
lmesg
lmesg.0
lmesg.1.gz
lmesg.2.gz
lmesg.3.gz
lmesg.4.gz
lpkg.log
faillog
fontconfig.log
fdm3

```

```
192.168.56.102 | CHANGED | rc=0 >>
alternatives.log
appport.log
apt
auth.log
auth.log.1
boot.log
boot.log.1
bootstrap.log
btmpt
cloud-init.log
cloud-init-output.log
cups
cups-browsed
dist-upgrade
dmesg
dmesg.0
dmesg.1.gz
dmesg.2.gz
dmesg.3.gz
dpkg.log
faillog
fontconfig.log
gdm3
gpu-manager.log
hp
installer
journal
wdjavillonar@workstation:~$ ansible all -m shell -a "cd /var/log/a
192.168.56.102 | CHANGED | rc=0 >>
eipp.log.xz
history.log
term.log
192.168.56.101 | CHANGED | rc=0 >>
eipp.log.xz
history.log
term.log
```

3. This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.
 - a. Issue the command: *ansible all -m apt -a name=snapd --become --ask-become-pass*

```
wdjavillonar@workstation:~$ ansible all -m apt -a name=snapd --become --ask-
BECOME password:
192.168.56.102 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787018507,
  "cache_updated": false,
  "changed": false
}
192.168.56.101 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787018499,
  "cache_updated": false,
  "changed": false
}
```

Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?

The command was successful on both remote servers. The state=latest option checks whether the latest available version of snapd is installed. Both servers already had the latest version, so Ansible reported changed: false, meaning no changes were necessary.

- b. Now, try to issue this command: *ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass*

```
wdjavillonar@workstation:~$ ansible all -m apt -a "name=snapd state=latest" --become
e-pass
BECOME password:
192.168.56.101 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787018499,
  "cache_updated": false,
  "changed": false
}
192.168.56.102 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787018507,
  "cache_updated": false,
  "changed": false
}
```

Describe the output of this command. Notice how we added the command state=latest and placed them in double quotations.

Task 2: Writing our First Playbook

1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (`CPE232_yourname`). Issue the command `nano install_apache.yml`. This will create a playbook file called `install_apache.yml`. The `.yaml` is the basic standard extension for playbook files.

```
wdjavillonar@workstation:~$ ls
ansible-lab    Desktop    Downloads  Pictures   snap       Videos
CPE232_Javillonar  Documents Music      Public     Templates
wdjavillonar@workstation:~$ cd CPE232_Javillonar
wdjavillonar@workstation:~/CPE232_Javillonar$ pwd
/home/wdjavillonar/CPE232_Javillonar
wdjavillonar@workstation:~/CPE232_Javillonar$
```

When the editor appears, type the following:

```
---
- hosts: all #change if needed
  become: true
  tasks:

    - name: install apache2 package
      apt:
        name: apache2
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ cat install_apache.yml
---
- host: all
  become: true

  tasks:
    - name: Install Apache2
      apt:
        name: apache2
```

Make sure to save the file. Take note also of the alignments of the texts. Use TAB instead of SPACE BAR for spacing.

2. Run the yml file using the command: *ansible-playbook --ask-become-pass install_apache.yml*. Describe the result of this command.
 - The playbook executed successfully on both remote servers. Ansible connected to each host and ensured that the apache2 package was installed. The PLAY RECAP showed no unreachable or failed hosts.

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook --ask-become-pass install_apache.
yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

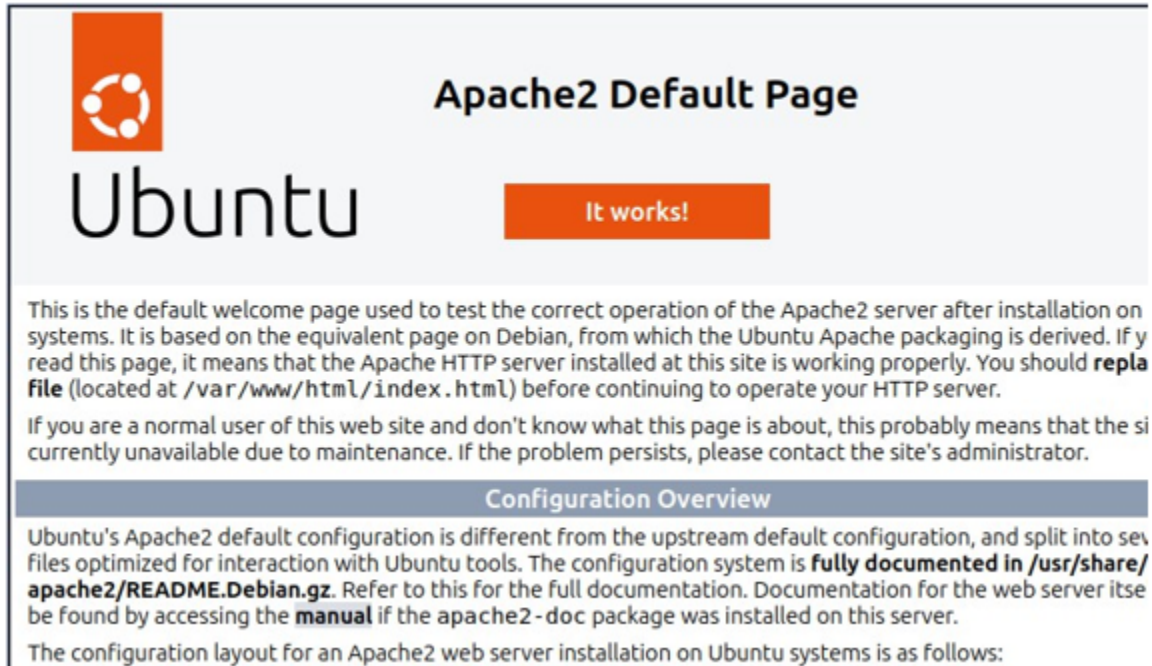
TASK [Install Apache2] *****
changed: [192.168.56.102]

changed: [192.168.56.101]

PLAY RECAP *****
192.168.56.101      : ok=2    changed=1    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
192.168.56.102      : ok=2    changed=1    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible all -m shell -a "systemctl is-active apache2"
192.168.56.102 | CHANGED | rc=0 >>
active
192.168.56.101 | CHANGED | rc=0 >>
active
```

3. To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.



Apache2 Default Page

Ubuntu

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace** file (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented in `/usr/share/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

FROM SERVER 1

serv1javillonar (Pings) [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Aug 18 13:50

apache2 Ubuntu Default Page x Firefox Privacy Notice — | x +

Not Secure http://192.168.56.101 80% ☆



Apache2 Default Page

Ubuntu

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

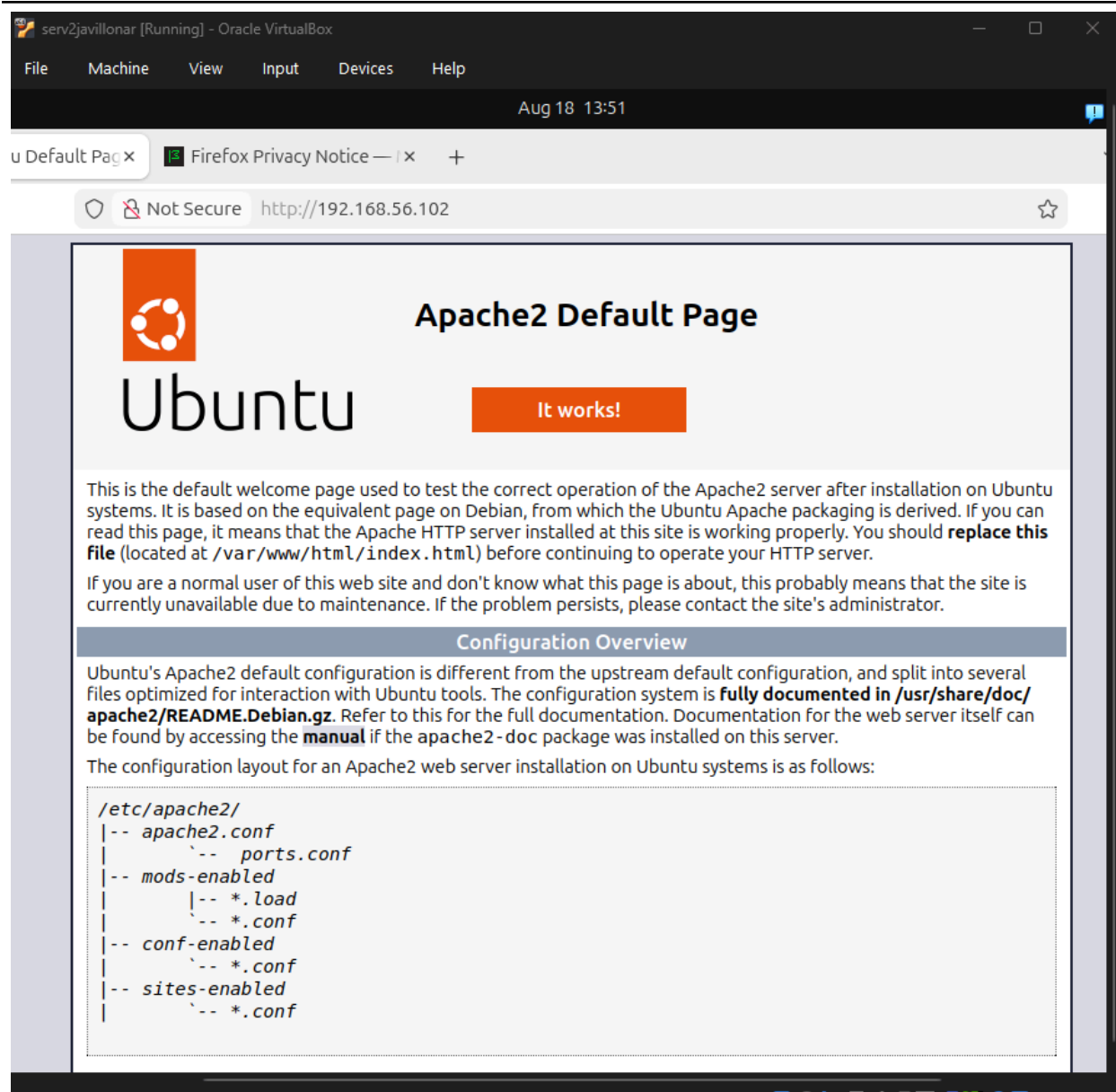
Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented in [/usr/share/doc/apache2/README.Debian.gz](#)**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

```
/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|-- mods-enabled
|   |-- *.load
|   |-- *.conf
|-- conf-enabled
|   |-- *.conf
|-- sites-enabled
|   |-- *.conf
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- `ports.conf` is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.
- Configuration files in the `mods-enabled/`, `conf-enabled/` and `sites-enabled/` directories contain particular configuration snippets which manage modules, global configuration fragments, or virtual host configurations, respectively.
- They are activated by symlinking available configuration files from their respective `*-available/` counterparts. These should be managed by using our helpers `a2enmod`, `a2dismod`, `a2ensite`, `a2dissite`, and

FROM SERVER 2



4. Try to edit the `install_apache.yml` and change the name of the package to any name that will not be recognized. What is the output?
5. This time, we are going to put additional tasks into our playbook. Edit the `install_apache.yml`. As you can see, we are now adding an additional command, which is the `update_cache`. This command updates existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.

```
--
- hosts: all #change if needed
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes

    - name: install apache2 package
      apt:
        name: apache2
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ cat install_apache.yml
```

```
---
- hosts: all
  become: true

  tasks:
    - name: Install Apache2
      apt:
        name: apache999
```

Save the changes to this file and exit.

6. Run the playbook and describe the output. Did the new command change anything on the remote servers? The playbook failed because apache999 is not a valid package available in the configured repositories. Ansible was unable to install the package and therefore marked the task as FAILED on the remote servers.

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook --ask-become-pass install_apache.
yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [Install Apache2] *****
fatal: [192.168.56.102]: FAILED! => {"changed": false, "msg": "No package matching 'apache999' i
s available"}
fatal: [192.168.56.101]: FAILED! => {"changed": false, "msg": "No package matching 'apache999' i
s available"}

PLAY RECAP *****
192.168.56.101      : ok=1    changed=0    unreachable=0    failed=1    skipped=0    resc
ued=0    ignored=0
192.168.56.102      : ok=1    changed=0    unreachable=0    failed=1    skipped=0    resc
ued=0    ignored=0
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ cat install_apache.yml
---
- hosts: all
  become: true

  tasks:
    - name: Update package cache
      apt:
        update_cache: yes
        name: apache2

    - name: Install Apache2
      apt:
        name: apache2
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook --ask-become-pass install_apache.
yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.102]
ok: [192.168.56.101]

TASK [Update package cache] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [Install Apache2] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

PLAY RECAP *****
192.168.56.101      : ok=3    changed=0    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
192.168.56.102      : ok=3    changed=0    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
```

7. Edit again the `install_apache.yml`. This time, we are going to add a PHP support for the apache package we installed earlier.

```
---
- hosts: all #change if needed
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes

    - name: install apache2 package
      apt:
        name: apache2

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ cat install_apache.yml
```

```
---
- hosts: all
  become: true

  tasks:
    - name: Update package cache
      apt:
        update_cache: yes
        name: apache2

    - name: Install PHP support for Apache
      apt:
        name: libapache2-mod-php
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook --ask-become-pass install_apache.
yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.102]
ok: [192.168.56.101]

TASK [Update package cache] *****
ok: [192.168.56.102]
ok: [192.168.56.101]

TASK [Install PHP support for Apache] *****

changed: [192.168.56.101]
changed: [192.168.56.102]

PLAY RECAP *****
192.168.56.101      : ok=3    changed=1    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
192.168.56.102      : ok=3    changed=1    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
```

Save the changes to this file and exit.

8. Run the playbook and describe the output. Did the new command change anything on the remote servers? The playbook successfully installed PHP support for Apache on both remote servers. The Install PHP support for Apache task reported changed, indicating that the PHP Apache module was installed. The PLAY RECAP showed failed=0 for both servers.
9. Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository.
https://github.com/MadhelKathlynAnnJavillonar/CPE232_Javillonar

```
wdjavillonar@workstation:~/CPE232_Javillonar$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  install_apache.yml

nothing added to commit but untracked files present (use "git add" to track)
wdjavillonar@workstation:~/CPE232_Javillonar$ git add install_apache.yml
wdjavillonar@workstation:~/CPE232_Javillonar$ git commit -m "Complete Apache and PHP Ansible playbook"
[main 440d865] Complete Apache and PHP Ansible playbook
 1 file changed, 14 insertions(+)
 create mode 100644 install_apache.yml
wdjavillonar@workstation:~/CPE232_Javillonar$ git branch --show-current
main
wdjavillonar@workstation:~/CPE232_Javillonar$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 460 bytes | 230.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:MadhelKathlynAnnJavillonar/CPE232_Javillonar.git
 09f8987..440d865  main -> main
wdjavillonar@workstation:~/CPE232_Javillonar$ git remote -v
origin  git@github.com:MadhelKathlynAnnJavillonar/CPE232_Javillonar.git (fetch)
origin  git@github.com:MadhelKathlynAnnJavillonar/CPE232_Javillonar.git (push)
wdjavillonar@workstation:~/CPE232_Javillonar$
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. What is the importance of using a playbook?

A playbook makes it easier to manage multiple servers because we can run several tasks automatically using one file. It also helps keep the server setup organized and consistent.

2. Summarize what we have done on this activity.

We used Ansible to manage the remote servers, installed and checked packages like Vim and Snapd, and created a playbook to install Apache and PHP. We also tested an invalid package, checked the APT logs, updated the package cache, and pushed our playbook to GitHub.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini and Claude

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Gemini for constructing my sentences correctly and Claude for verifying correct commands.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.



COMPUTER
ENGINEERING

QUEZON CITY

Extent of Use: I can confirm that this activity was my original work and AI didn't replace my own learnings and ideas.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.